

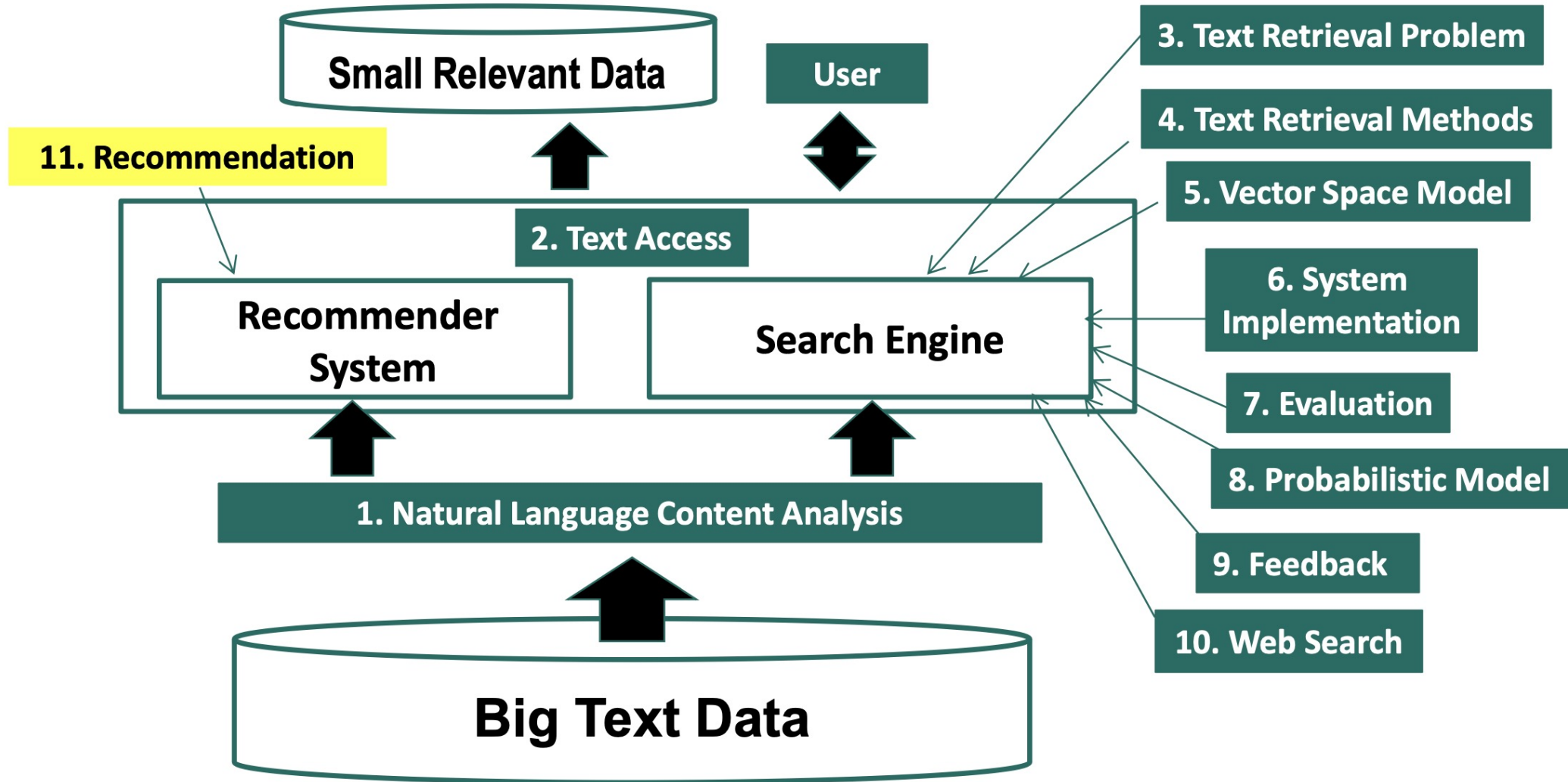


Fundamentals of Big Data Analytics

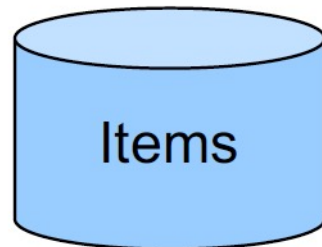
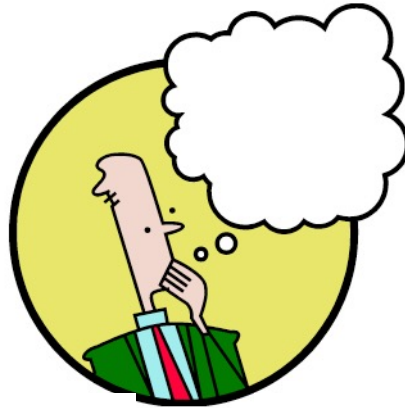
Recommender Systems

Dr. Iqra Safder

Recommender Systems



Recommendations



Examples:

amazon.com



m o v i e l e n s
helping you find the *right* movies

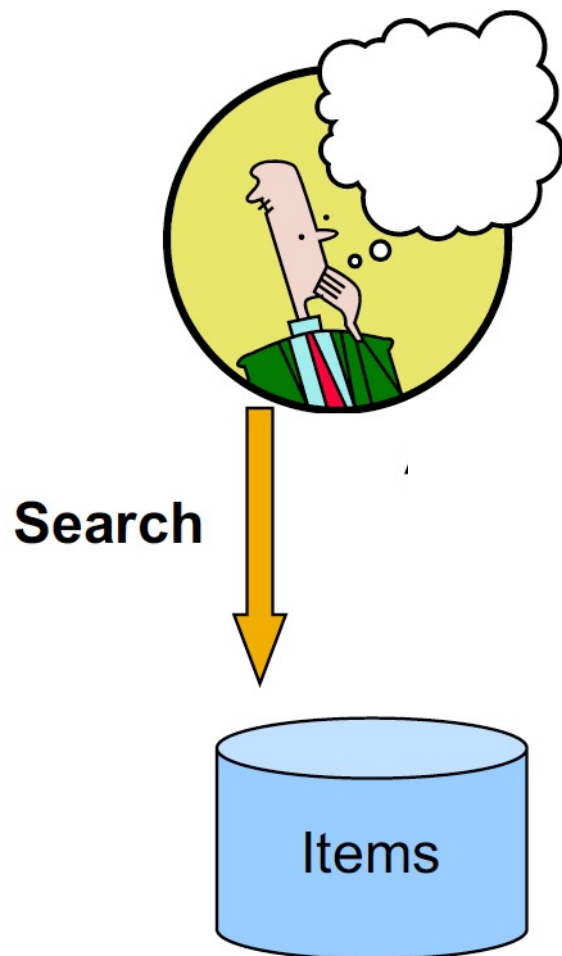
last.fm™
the social music revolution

Google™
News

You Tube

XBOX
LIVE

Recommendations



Examples:

amazon.com



m o v i e l e n s
helping you find the *right* movies

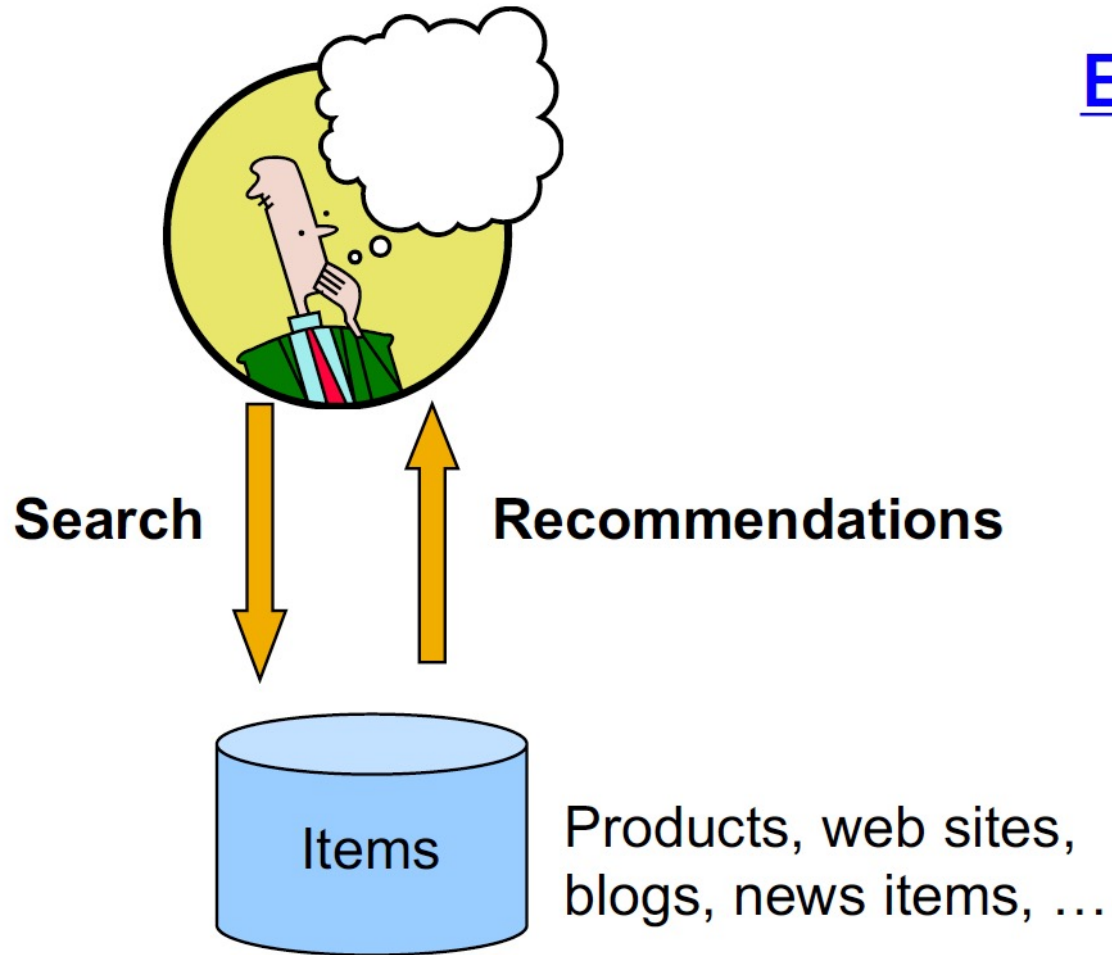
last.fm™
the social music revolution

Google™
News

You Tube

XBOX
LIVE

Recommendations



Examples:

amazon.com



StumbleUpon



del.icio.us



movie lens
helping you find the *right* movies

last.fm™
the social music revolution

Google™
News

YouTube

XBOX
LIVE

From Scarcity to Abundance

- **Shelf space is a scarce commodity for traditional retailers**
 - Also: TV networks, movie theaters,...
- **Web enables near-zero-cost dissemination of information about products**
 - From scarcity to abundance

Types of Recommendations

- More choice necessitates better filters
 - Recommendation engines
 - How **Into Thin Air** made **Touching the Void** a bestseller (<http://www.wired.com/wired/archive/12.10/tail.html>)
- Examples
 - Books, movies, music, news articles
 - People (friend recommendations on Facebook, LinkedIn, and Twitter)

Formal Model

- X = set of **Customers**
- S = set of **Items**
- **Utility function** $u: X \times S \rightarrow R$
 - R = set of ratings
 - R is a totally ordered set
 - e.g., **0-5** stars, real number in **[0,1]**

Utility Matrix

	Avatar	LOTR	Matrix	Pirates
Alice	1		0.2	
Bob		0.5		0.3
Carol	0.2		1	
David				0.4

Key Problems

- **(1) Gathering “known” ratings for matrix**
 - How to collect the data in the utility matrix

Key Problems

- **(1) Gathering “known” ratings for matrix**
 - How to collect the data in the utility matrix
- **(2) Extrapolate unknown ratings from the known ones**
 - Mainly interested in high unknown ratings

1) Gathering Ratings

- **Explicit**

- Ask people to rate items
- Doesn't work well in practice – people can't be bothered
- Crowdsourcing: Pay people to label items

1) Gathering Ratings

■ Explicit

- Ask people to rate items
- Doesn't work well in practice – people can't be bothered
- Crowdsourcing: Pay people to label items

■ Implicit

- Learn ratings from user actions
 - E.g., purchase implies high rating
- What about low ratings?

2) Extrapolating Utilities

- **Key problem:** Utility matrix U is **sparse**
 - Most people have not rated most items
 - **Cold start:**
 - New items have no ratings
 - New users have no history
- **Approaches to recommender systems**
 - **1)** Content-based
 - **2)** Collaborative



User-User Collaborative Filtering

Item-Item Collaborative Filtering

Similar Users

	HP1	HP2	HP3	TW	SW1	SW2	SW3
<i>A</i>	4			5	1		
<i>B</i>	5	5	4				
<i>C</i>				2	4	5	
<i>D</i>		3					3

- Consider users $\mathbf{x}$ and $\mathbf{y}$ with rating vectors $\mathbf{r}_x$ and $\mathbf{r}_y$
- We need a similarity metric $\text{sim}(\mathbf{x}, \mathbf{y})$
- Capture intuition that $\text{sim}(A, B) > \text{sim}(A, C)$

Option 1: Jaccard Similarity

	HP1	HP2	HP3	TW	SW1	SW2	SW3
<i>A</i>	4			5	1		
<i>B</i>	5	5	4				
<i>C</i>				2	4	5	
<i>D</i>		3					3

- $\text{sim}(A,B) = |r_A \cap r_B| / |r_A \cup r_B|$
- $\text{sim}(A,B) = 1/5$; $\text{sim}(A,C) = 2/4$
 - $\text{sim}(A,B) < \text{sim}(A,C)$

Option 2: Cosine Similarity

	HP1	HP2	HP3	TW	SW1	SW2	SW3
<i>A</i>	4			5	1		
<i>B</i>	5	5	4				
<i>C</i>				2	4	5	
<i>D</i>		3					3

- $\text{sim}(A,B) = \cos(r_A, r_B)$
- $\text{sim}(A,B) = 0.38$, $\text{sim}(A,C) = 0.32$
 - $\text{sim}(A,B) > \text{sim}(A,C)$, but not by much

Option 2: Cosine Similarity

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4	0	0	5	1	0	0
B	5	5	4	0	0	0	0
C				2	4	5	
D		3					3

■ $\text{sim}(A,B) = \cos(r_A, r_B)$

■ $\text{sim}(A,B) = 0.38, \text{sim}(A,C) = 0.32$

■ $\text{sim}(A,B) > \text{sim}(A,C)$, but not by much

■ Problem: treats missing ratings as negative

- Problem with cosine:
 - C really loves SW
 - A hates SW
 - B just hasn't seen it
- Another problem: we'd like to normalize the raters
 - D rated everything the same; not very useful

Option 3: Centered Cosine

- Normalize ratings by subtracting row mean

	HP1	HP2	HP3	TW	SW1	SW2	SW3
<i>A</i>	4			5	1		
<i>B</i>	5	5	4				
<i>C</i>				2	4	5	
<i>D</i>		3					3

Option 3: Centered Cosine

- Normalize ratings by subtracting row mean

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

10/3
14/3

Option 3: Centered Cosine

- Normalize ratings by subtracting row mean

	HP1	HP2	HP3	TW	SW1	SW2	SW3
<i>A</i>	4			5	1		
<i>B</i>	5	5	4				
<i>C</i>				2	4	5	
<i>D</i>		3					3

	HP1	HP2	HP3	TW	SW1	SW2	SW3
<i>A</i>	$2/3$			$5/3$	$-7/3$		
<i>B</i>	$1/3$	$1/3$	$-2/3$				
<i>C</i>				$-5/3$	$1/3$	$4/3$	
<i>D</i>		0					0

Option 3: Centered Cosine Similarity

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	2/3			5/3	-7/3		
B	1/3	1/3	-2/3				
C				-5/3	1/3	4/3	
D		0					0

- $\text{sim}(A,B) = \cos(r_A, r_B) = 0.09$; $\text{sim}(A,C) = -0.56$
 - $\text{sim}(A,B) > \text{sim}(A,C)$
- Captures intuition better
 - Missing ratings treated as “average”
 - Handles “tough raters” and “easy raters”

Item-Item Collaborative Filtering

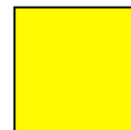
- So far: **User-user collaborative filtering**
- **Another view: Item-item**
 - For item i , find other similar items
 - Estimate rating for item i based on ratings for similar items
 - Can use same similarity metrics and prediction functions as in user-user model

Item-Item Collaborative Filtering

		users											
		1	2	3	4	5	6	7	8	9	10	11	12
movies	1	1		3			5			5		4	
	2			5	4			4			2	1	3
	3	2	4		1	2		3		4	3	5	
	4		2	4		5			4			2	
	5			4	3	4	2					2	5
	6	1		3		3			2			4	



- unknown rating



- rating between 1 to 5

Item-Item Collaborative Filtering

		users											
		1	2	3	4	5	6	7	8	9	10	11	12
movies	1	1		3		?	5			5		4	
	2			5	4			4			2	1	3
	3	2	4		1	2		3		4	3	5	
	4		2	4		5			4			2	
	5			4	3	4	2					2	5
	6	1		3		3			2			4	



- estimate rating of movie 1 by user 5

→

	users												
	1	2	3	4	5	6	7	8	9	10	11	12	$\text{sim}(1,m)$
movies	1	1		3		?	5			5		4	1.00
	2			5	4			4			2	1	3
	<u>3</u>	2	4		1	2		3		4	3	5	?
	4		2	4		5			4			2	..
	5			4	3	4	2					2	5
	<u>6</u>	1		3		3			2			4	..

→ **Neighbor selection:**
Identify movies similar to
movie 1, rated by user 5

Here we use mean centered item-overlap cosine as similarity:
1) Subtract mean rating m_i from each movie i between rows
2) Compute (item-overlapping) cosine similarities

		users											
		1	2	3	4	5	6	7	8	9	10	11	12
movies	1	1		3		?	5			5		4	
	Subtract mean rating m_i from each movie i $m_1 = (1+3+5+5+4)/5 = 18/5$												
		1	2	3	4	5	6	7	8	9	10	11	12
	1	-13/5		-3/5		?	7/5			7/5		2/5	
	3	-1	1		-2	-1		0		1	0	2	
	6	-8/5		2/5		2/5			-3/5			7/5	

Showing computation only for #3 and #6

Neighbor selection:

Identify movies similar to movie 1, rated by user 5

Here we use mean centered item-overlap cosine as similarity:

- 1) Subtract mean rating m_i from each movie i
- 2) Compute (item-overlapping) cosine similarities between rows

	users												
	1	2	3	4	5	6	7	8	9	10	11	12	sim(1,m)
movies	1	-13/5		-3/5	?	7/5			7/5		2/5		1.00
	2			5	4		4			2	1	3	..
	<u>3</u>	-1	1		-2	-1	0		1	0	2		?
	4		2	4		5		4			2		..
	5			4	3	4	2				2	5	..
	<u>6</u>	-8/5		2/5		2/5		-3/5			7/5		?

Neighbor selection:

Identify movies similar to movie 1, rated by user 5

Here we use mean centered item-overlap cosine as similarity:

- 1) Subtract mean rating m_i from each movie i
- 2) Compute (item-overlapping) cosine similarities between rows

	users												
	1	2	3	4	5	6	7	8	9	10	11	12	sim(1,m)
movies	1	-13/5	-3/5		?	7/5			7/5		2/5		1.00
	2		5	4			4			2	1	3	..
	<u>3</u>	-1	1	-2	-1		0		1	0	2		?
	4		2	4	5			4			2		..
	5		4	3	4	2					2	5	..
	<u>6</u>	-8/5	2/5		2/5			-3/5			7/5		?

Neighbor selection:

Identify movies similar to movie 1, rated by user 5

Here we use mean centered item-overlap cosine as similarity:

- 1) Subtract mean rating m_i from each movie i
- 2) Compute (item-overlapping) cosine similarities between rows

	users												
	1	2	3	4	5	6	7	8	9	10	11	12	sim(1,m)
movies	1	-13/5	-3/5		?	7/5			7/5		2/5		1.00
	2		5	4			4			2	1	3	..
	3	-1	1	-2	-1		0		1	0	2		?
	4		2	4	5			4			2		..
	5		4	3	4	2					2	5	..
	6	-8/5	2/5		2/5			-3/5			7/5		?

Neighbor selection:

Identify movies similar to movie 1, rated by user 5

Here we use mean centered item-overlap cosine as similarity:

- 1) Subtract mean rating m_i from each movie i
- 2) Compute (item-overlapping) cosine similarities between rows

	users												
	1	2	3	4	5	6	7	8	9	10	11	12	sim(1,m)
movies	1	-13/5	-3/5		?	7/5			7/5		2/5		1.00
	2		5	4			4			2	1	3	..
	3	-1	1	-2	-1		0		1	0	2		?
	4		2	4	5			4			2		..
	5		4	3	4	2					2	5	..
	6	-8/5	2/5		2/5			-3/5			7/5		?

Neighbor selection:

Identify movies similar to movie 1, rated by user 5

Here we use mean centered item-overlap cosine as similarity:

- 1) Subtract mean rating m_i from each movie i
- 2) Compute (item-overlapping) cosine similarities between rows

Compute Cosine Similarity:

For rows 1 and 3, they both have values for users 1, 9 and 11.

$$\text{sim}(1, 3) = \frac{(-13/5)(-1) + (7/5)(1) + (2/5)(2)}{\sqrt{(-13/5)^2 + (7/5)^2 + (2/5)^2} \cdot \sqrt{(-1)^2 + (1)^2 + (2)^2}} \approx 0.658$$

For rows 1 and 6, they both have values for users 1, 3 and 11.

$$\text{sim}(1, 6) = \frac{(-13/5)(-8/5) + (-3/5)(2/5) + (2/5)(7/5)}{\sqrt{(-13/5)^2 + (-3/5)^2 + (2/5)^2} \cdot \sqrt{(-8/5)^2 + (2/5)^2 + (7/5)^2}} \approx 0.768$$

Stanford

		users													
		1	2	3	4	5	6	7	8	9	10	11	12		
movies	1	1		3		?	5			5		4		$\text{sim}(1,m)$	1.000
	2			5	4			4			2	1	3	..	
	<u>3</u>	2	4		1	2		3		4	3	5		<u>.658</u>	←
	4		2	4		5			4			2		..	
	5			4	3	4	2					2	5	..	
	<u>6</u>	1		3		3			2			4		<u>.768</u>	←

Compute similarity weights:

$s_{1,3}=.658$, $s_{1,6}=.768$ (we compute $s_{1,2}$, $s_{1,4}$, $s_{1,5}$ too; let's assume those are smaller)

Slides adapted from Jure Leskovec, CS246 and J. Leskovec, A. Rajaraman, J. Ullman: *Mining of Massive Datasets*

Stanford

		users												
		1	2	3	4	5	6	7	8	9	10	11	12	sim(1,m)
movies	1	1		3		2.54	5			5		4		1.000
	2			5	4			4			2	1	3	..
	<u>3</u>	2	4		1	2		3		4	3	5		<u>.658</u>
	4		2	4		5			4			2		..
	5			4	3	4	2					2	5	..
	<u>6</u>	1		3		3			2			4		<u>.768</u>

Predict by taking weighted average:

$$r_{1,5} = (0.658 \cdot 2 + 0.768 \cdot 3) / (0.658 + 0.768) = 2.54$$

$$r_{ix} = \frac{\sum_{j \in N(i,x)} s_{ij} r_{jx}}{\sum s_{ij}}$$

Slides adapted from Jure Leskovec, CS246 and J. Leskovec, A. Rajaraman, J. Ullman: *Mining of Massive Datasets*

Item-Item vs. User-User

- In theory, user-user and item-item are dual approaches
- In practice, item-item outperforms user-user in many use cases
- Items are “simpler” than users
 - Items belong to a small set of “genres”, users have varied tastes
 - Item Similarity is more meaningful than User Similarity

Pros/Cons of Collaborative Filtering

+ Works for any kind of item

- No feature selection needed

- Cold Start:

- Need enough users in the system to find a match

- Sparsity:

- The user/ratings matrix is sparse
- Hard to find users that have rated the same items

- First rater:

- Cannot recommend an item that has not been previously rated

- Popularity bias:

- Cannot recommend items to someone with unique taste
- Tends to recommend popular items

- Ethical and social issues:

- Can lead to filter bubbles and radicalization spirals

Stanford

Spark Alternating Least Squares (ALS) for recommender systems

```
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.recommendation import ALS
from pyspark.sql import Row

lines = spark.read.text("data/mllib/als/sample_movielens_ratings.txt").rdd
parts = lines.map(lambda row: row.value.split(":"))
ratingsRDD = parts.map(lambda p: Row(userId=int(p[0]), movieId=int(p[1]),
                                     rating=float(p[2]), timestamp=int(p[3])))
ratings = spark.createDataFrame(ratingsRDD)
(training, test) = ratings.randomSplit([0.8, 0.2])

# Build the recommendation model using ALS on the training data
# Note we set cold start strategy to 'drop' to ensure we don't get NaN evaluation metrics
als = ALS(maxIter=5, regParam=0.01, userCol="userId", itemCol="movieId", ratingCol="rating",
          coldStartStrategy="drop")
model = als.fit(training)

# Evaluate the model by computing the RMSE on the test data
predictions = model.transform(test)
evaluator = RegressionEvaluator(metricName="rmse", labelCol="rating",
                                predictionCol="prediction")
rmse = evaluator.evaluate(predictions)
print("Root-mean-square error = " + str(rmse))

# Generate top 10 movie recommendations for each user
userRecs = model.recommendForAllUsers(10)
# Generate top 10 user recommendations for each movie
movieRecs = model.recommendForAllItems(10)

# Generate top 10 movie recommendations for a specified set of users
users = ratings.select(als.getUserCol()).distinct().limit(3)
userSubsetRecs = model.recommendForUserSubset(users, 10)
# Generate top 10 user recommendations for a specified set of movies
movies = ratings.select(als.getItemCol()).distinct().limit(3)
movieSubSetRecs = model.recommendForItemSubset(movies, 10)
```

<https://spark.apache.org/docs/latest/ml-collaborative-filtering.html>

Other Resources

- <https://towardsdatascience.com/prototyping-a-recommender-system-step-by-step-part-2-alternating-least-square-als-matrix-4a76c58714a1>
- https://link.springer.com/chapter/10.1007/978-3-540-68880-8_32